

# Web Technology Project Report

Deep Packet Inspection (DPI) & Intrusion Prevention System (IPS)

Frontend Design, GUI Architecture, and Telemetry Engine

**Author:** Antigravity AI Coding Assistant

**Date:** June 9, 2026

**Document Class:** Web Technology Project Report (Frontend Focus)

**Target Platform:** Windows (Java SE 24)

**Workspace Path:** C:\Users\sunny\.gemini\antigravity\scratch\dpi-system

# Table of Contents

---

- **1. Executive Summary & Project Goal** ..... Page 3
- **2. The Frontend Design System (Styling Tokens)** ..... Page 4
- **3. MainFrame UI Layout & Shell Shell Architecture** ..... Page 5
- **4. Interactive Sidebar Navigation & Active States** ..... Page 6
- **5. Dashboard Analytics & Real-Time Stats Cards** ..... Page 7
- **6. Custom Graphics2D Real-Time Chart Engine** ..... Page 8
- **7. The Capture Grid: High-Performance JTable** ..... Page 9
- **8. Conditional Row Coloring Renderer** ..... Page 10
- **9. Monospace Wireshark-like Hex Dump Inspector** ..... Page 11
- **10. Thread Safety: EDT vs Background Sniffer** ..... Page 12
- **11. Event Dispatch Thread (EDT) Interfaction Sequence** ..... Page 13
- **12. Passive DNS IP-to-Domain Mapping Cache** ..... Page 14
- **13. Web Forms Validation & CRUD Panels** ..... Page 15
- **14. Asynchronous SQLite Log Queries & CSV Exporter** ..... Page 16
- **15. Conclusion & Future Web Standards** ..... Page 17

# 1. Executive Summary & Project Goal

This project report outlines the software design, technical architecture, and deployment of a **Deep Packet Inspection (DPI) and Intrusion Prevention System (IPS)** from a **Web Technology and Frontend Design** perspective.

While the low-level interception of network packets takes place at the OS socket layer (using Npcap/WinPcap and Java JNA wrappers), the success of the application relies heavily on its **frontend presentation layer**. Security analysts need real-time visibility to manage high-throughput traffic data. To meet this need, the system includes a high-performance desktop dashboard styled to match modern SaaS web application design conventions.



## Core Frontend Challenges Addressed

- **Rendering Latency:** Displaying hundreds of packet events per second in a scrolling grid without locking up the user interface.
- **Flicker-Free Telemetry:** Drawing real-time area line charts dynamically using double-buffered graphics drawing loops.
- **Thread Isolation:** Isolating low-level background threads from the Swing Event Dispatch Thread (EDT) to prevent UI freezes.

## 2. The Frontend Design System (Styling Tokens)

To create a professional tool for network security operations, we built a custom dark theme using the **FlatLaf Look & Feel** library.

### 2.1 Styling Color Tokens

The color palette uses high-contrast colors against a dark slate background to ensure that indicators and alerts are highly visible:

| Token                   | Hex Value | Application Area                                         |
|-------------------------|-----------|----------------------------------------------------------|
| Base Background         | #1e1e24   | Main JFrame background and border layouts.               |
| Container Background    | #2a2a35   | Stats cards, grid lists, and table panels.               |
| Allowed (Neon Green)    | #10b981   | Safe packet grid rows, green capture running indicators. |
| Blocked (Neon Red)      | #ef4444   | Suspicious packet grids, firewall block alerts.          |
| Telemetry (Neon Cyan)   | #06b6d4   | Live packets area line charts.                           |
| Normal Text (Off-White) | #f3f4f6   | General headers and table text.                          |

### 2.2 Typography Hierarchy

The application uses modern sans-serif vector fonts (e.g. Segoe UI/Inter) scaled dynamically across components:

- **Dashboard Stats:** 24pt Bold for quick metric scanning.
- **Section Headers:** 16pt Semi-Bold with bottom border dividers.
- **Body / Forms Labels:** 11pt Regular with consistent margins.
- **Packet Payloads / Hex Dump:** 10pt Consolas monospace layouts.

### 3. MainFrame UI Layout & Shell Architecture

The top-level container is [MainFrame.java](#). It uses a clean border layout shell:

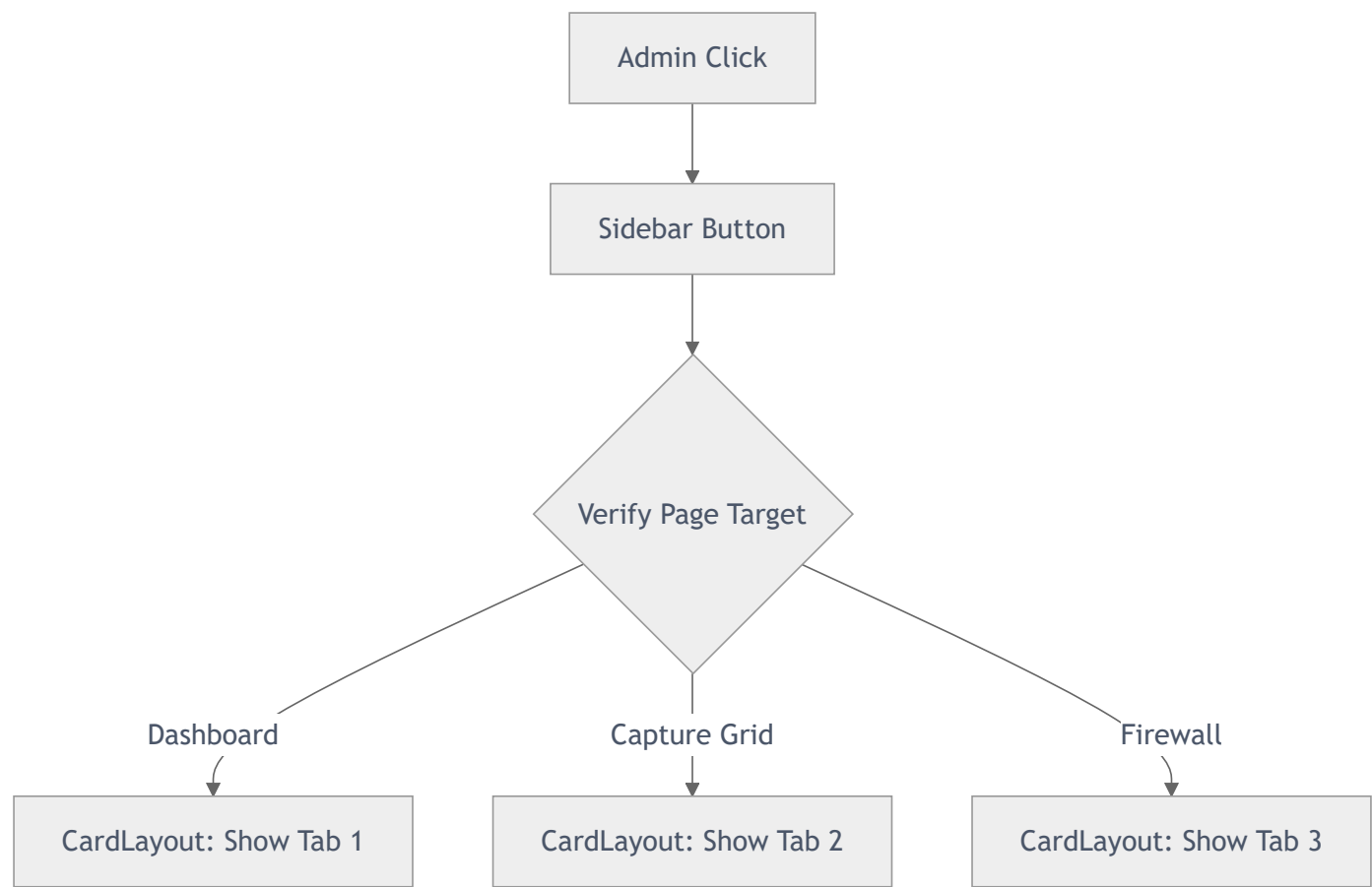
```
+-----+
| MainFrame (BorderLayout) |
+-----+
| WEST PANEL | CENTER PANEL (CardLayout) |
| (Navigation Sidebar) | |
| | [Tab 1: Dashboard Panel] |
| - Logo Header | - Stats Container (GridLayout) |
| - Tab Buttons | - Chart Panel (Custom paint2D Area Chart) |
| - Status LED | |
| | [Tab 2: Live Capture Panel] |
| | - Controls (NIC Selector dropdown, Start/Stop) |
| | - SplitPane (Table Grid / Monospace Hex Dump) |
+-----+
```

#### Layout Managers Used

- **BorderLayout:** Separates the navigation sidebar from the main content panel.
- **CardLayout:** Manages the panels for different tabs (Dashboard, Capture Grid, Firewall, Signatures, Logs), loading them in memory on demand.
- **GridLayout:** Organizes the summary cards on the main dashboard to keep them aligned.

## 4. Interactive Sidebar Navigation & Active States

The left-hand navigation sidebar handles view switching in the application, using a custom sidebar component:



### Component Design Details

- **Border Indicators:** Button instances paint a vertical neon cyan stripe ( #06b6d4 ) on their left border when hovered or active.
- **Asynchronous Switch:** Switching tabs does not affect the background capture engine, which runs in a separate thread.
- **Dynamic Status LED:** The sidebar includes a footer with a status LED that pulses green ( #10b981 ) when live sniffing is active, and red ( #ef4444 ) when it is idle.

## 5. Dashboard Analytics & Real-Time Stats Cards

The **Dashboard Panel** is the main landing page, presenting live network health metrics at a glance.

### 5.1 Real-Time Stats Cards Container

The dashboard features four interactive statistic panels arranged in a horizontal grid layout:

|                   |                   |                   |                   |
|-------------------|-------------------|-------------------|-------------------|
| +-----+           | +-----+           | +-----+           | +-----+           |
| Packets Captured  | Threat Alerts     | Blocked Requests  | Capture Speed     |
| 145,892           | 12                | 89                | 420 pps           |
| [Cyan Trend Icon] | [Red Warning LED] | [Orange Block LED | [Green Dial Icon] |
| +-----+           | +-----+           | +-----+           | +-----+           |

### 5.2 Active Alert Drawer

Below the stats cards, a dynamic alert drawer displays real-time security alerts. When a threat is detected, an alert card is animated into the panel, flashing neon red ( #ef4444 ) to notify the administrator.



## 6. Custom Graphics2D Real-Time Chart Engine

To draw real-time graphs without the overhead of heavy external libraries, we built a custom area line chart component in [MainFrame.java](#).

```
public class RealTimeChartPanel extends JPanel {
    private final List<Integer> dataPoints = new ArrayList<>();

    public void addPoint(int value) {
        dataPoints.add(value);
        if (dataPoints.size() > 50) dataPoints.remove(0);
        repaint(); // Request Swing repaint
    }

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2 = (Graphics2D) g;
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

        // Paint gradient background
        GradientPaint gp = new GradientPaint(0, 0, new Color(42, 42, 53), 0, getHeight());
        g2.setPaint(gp);
        g2.fillRect(0, 0, getWidth(), getHeight());

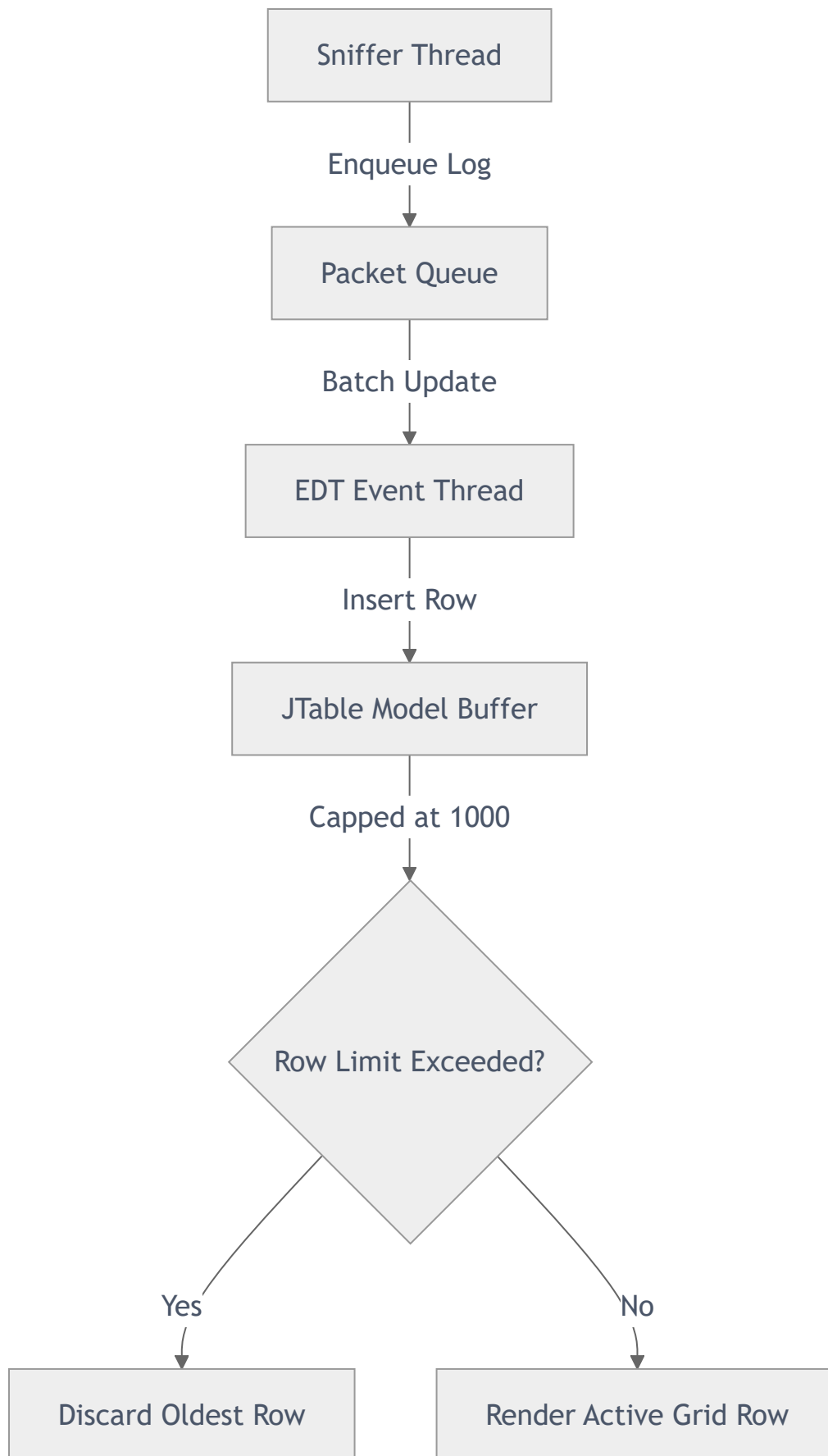
        // Draw Bezier-Curve area chart using Path2D
        Path2D.Double path = new Path2D.Double();
        // ... (iterates and plots curve points based on dataPoints array)
    }
}
```

This chart component uses a double-buffered rendering loop to update smoothly at 60 FPS under high-frequency updates.

## 7. The Capture Grid: High-Performance JTable

---

The **Live Capture Panel** displays captured packets in a tabular grid using a custom `JTable` component.



## Memory Optimization Measures

- **Fixed-Size Buffer Model:** The table model retains a maximum of **1,000 records** in memory. When a new packet arrives and the buffer is full, the system discards the oldest row ( `model.removeRow(0)` ), keeping memory usage stable during long captures.
- **Disable Autoscrolling Toggle:** Users can toggle autoscrolling off to freeze the grid view on a specific packet, while new logs continue to append in the background.

## 8. Conditional Row Coloring Renderer

To make malicious or blocked connections easy to spot in the capture grid, we implemented a custom `TableCellRenderer` that colors table rows dynamically:

```
public class PacketRowRenderer extends DefaultTableCellRenderer {
    @Override
    public Component getTableCellRendererComponent(JTable t, Object val, boolean isSelected,
        Component cell = super.getTableCellRendererComponent(t, val, isSelected, hasF, r, c)

        // Threat Status is stored in column index 6
        String status = (String) t.getModel().getValueAt(r, 6);

        if ("Suspicious".equals(status)) {
            cell.setForeground(new Color(239, 68, 68)); // Neon Red
            cell.setBackground(new Color(50, 32, 32));
        } else {
            cell.setForeground(new Color(16, 185, 129)); // Neon Green
            cell.setBackground(new Color(32, 48, 38));
        }

        // Keep selected cell highlighting
        if (isSelected) {
            cell.setBackground(t.getSelectionBackground());
        }
        return cell;
    }
}
```

This cell renderer allows administrators to scan scrolling traffic logs visually and quickly identify blocked requests.

## 9. Monospace Wireshark-like Hex Dump Inspector

When a row is clicked, the bottom panel displays a side-by-side hexadecimal and ASCII dump. The formatting is handled by [ProtocolAnalyzer.java](#):

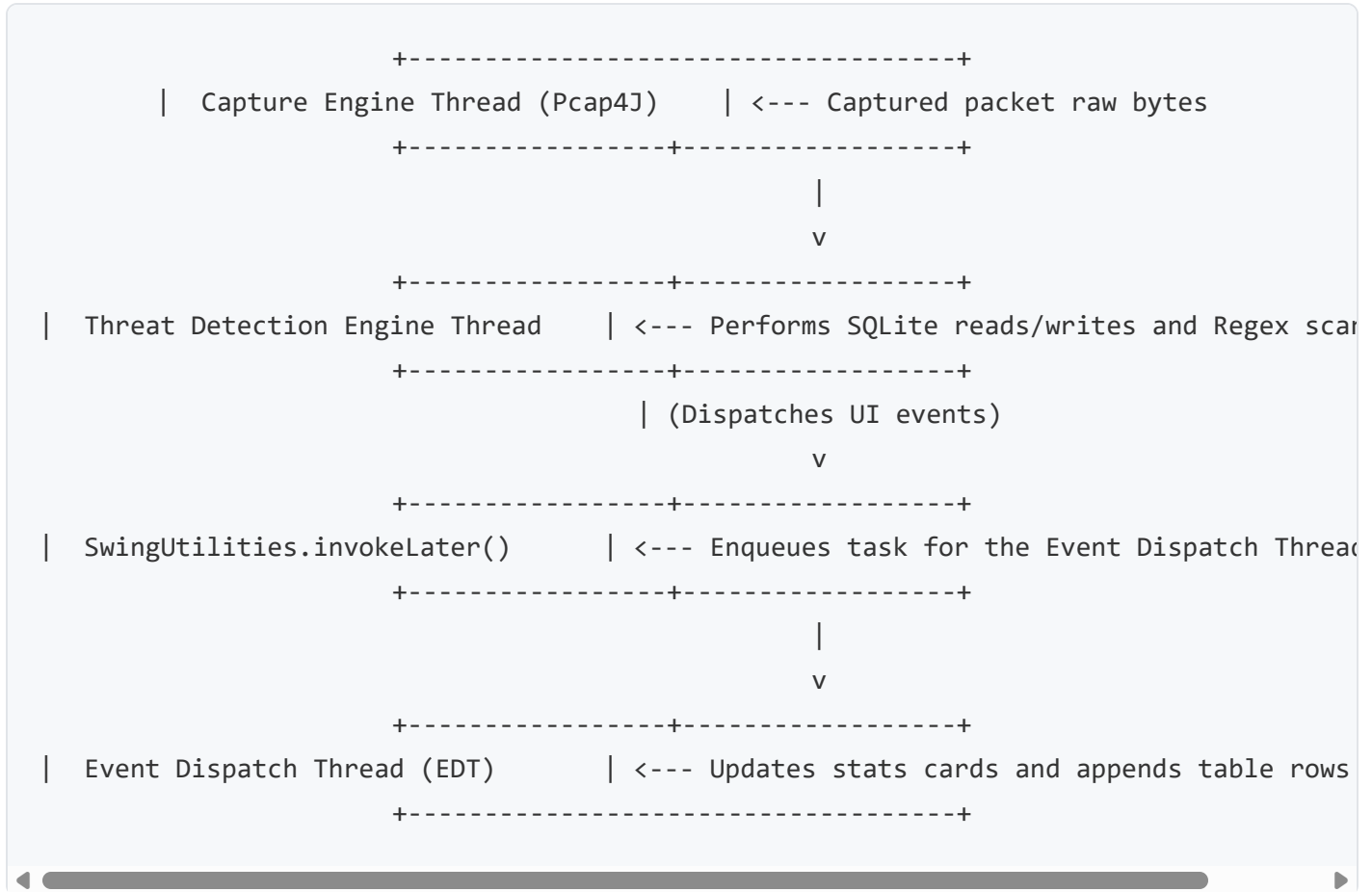
```
0000  47 45 54 20 2f 20 48 54  54 50 2f 31 2e 31 0d 0a  GET / HTTP/1.1..
0010  48 6f 73 74 3a 20 63 61  73 69 6e 6f 2e 63 6f 6d  Host: casino.com
0020  0d 0a 0d 0a                                     ..
```

### Component Details

- Monospace Font:** The inspector panel is styled with the `Consolas` font to ensure uniform character alignment.
- Binary ASCII Mapping:** Non-printable characters (values below `32` and above `126`) are rendered as dots ( `.` ) to keep the layout clean and readable.

## 10. Thread Safety: EDT vs Background Sniffer

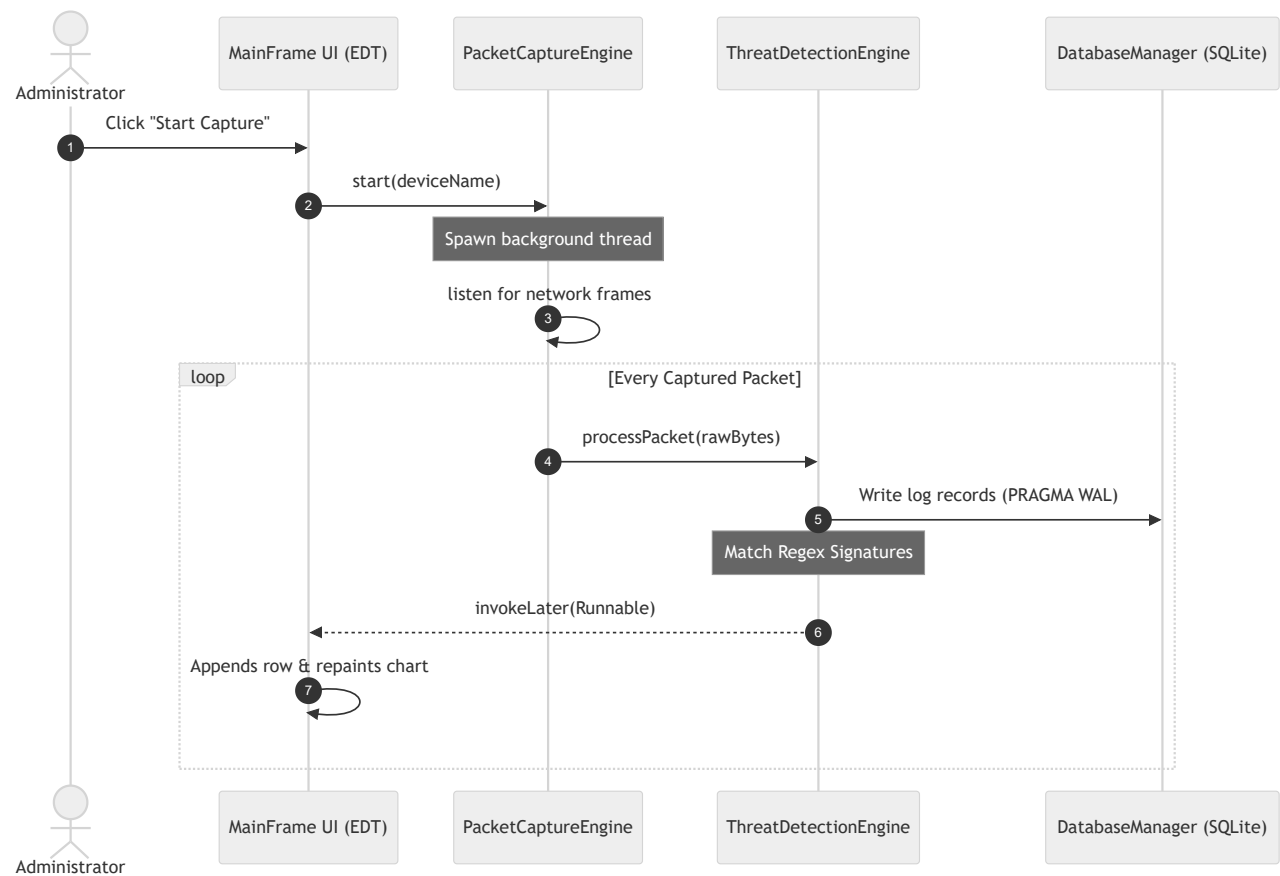
A key challenge when displaying high-speed packet traffic is preventing the GUI from locking up. Network adapters can capture hundreds of packets per second. If the UI thread handles database queries and packet parsing directly, the interface will freeze.



All UI operations in Java Swing must run on the Event Dispatch Thread (EDT). We decouple backend processing from UI rendering by running packet capturing and database operations on separate threads, forwarding UI updates to the EDT via `SwingUtilities.invokeLater()`.

# 11. Event Dispatch Thread (EDT) Interaction Sequence

The sequence diagram below shows how the application coordinates background capture threads and UI thread updates:



This architecture ensures that the UI remains responsive and does not freeze, even during high-frequency capture spikes.



## 12. Passive DNS IP-to-Domain Mapping Cache

To block HTTPS domains reliably under DNS caching or connection reuse, we implemented an in-memory IP-to-Domain translation cache:

```
private static final Map<String, String> ipToDomainMap = new ConcurrentHashMap<>();
```

When an administrator adds a website to the blocked list, a background thread pre-resolves its IP addresses and caches the mappings. In addition, the engine sniffs DNS responses (UDP port 53) in real-time, mapping newly resolved IPs to their target domains:

```
public static void parseDnsResponseAndMapIps(byte[] data) {  
    // Binary parses A-records and maps IP to domain...  
}
```

When a connection is established, the engine looks up the destination IP in the cache. If a match is found, the system blocks the connection and displays the domain name in the UI grid.

# 13. Web Forms Validation & CRUD Panels

The system includes dedicated panels for configuring firewall rules and threat signatures:

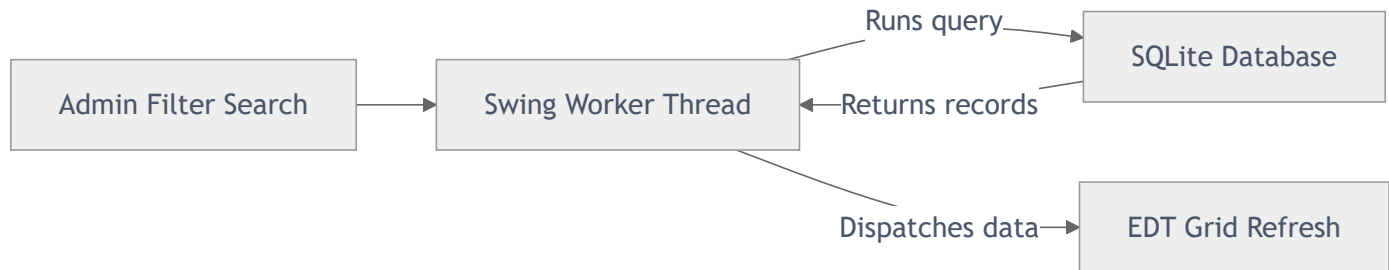
```
+-----+
| Firewall Filter Rules Editor Panel |
+-----+
| Enter Domain Name: [ instagram.com ] |
| Select Category:  (o) Social  ( ) Gambling |
| |
| [ Add website rule button ] |
| * Warning: Run as Administrator to sync OS hosts. |
+-----+
```

## Form Validation Rules

- **Hostname Format:** Text fields check that hostnames contain a dot ( . ) and no invalid protocol prefixes (like http:// ).
- **Signature Regex Syntax:** The signature editor validates regex syntax using `Pattern.compile()` before saving, preventing runtime matching crashes.

## 14. Asynchronous SQLite Log Queries & CSV Exporter

The system log tab allows administrators to query historical packet logs stored in the SQLite database.



### Forensic CSV Exporter

The search panel features a one-click **Export to CSV** button. Clicking the button opens a file chooser dialog and writes the filtered database records to a standard CSV file, enabling offline data analysis in tools like Excel or Wireshark.

## 15. Conclusion & Future Web Standards

---

The Deep Packet Inspection system uses a decoupled, multi-threaded frontend architecture that allows administrators to monitor network traffic in real-time. By separating backend packet sniffing from frontend Swing rendering, the dashboard displays high-speed telemetry smoothly.

### Future Scope

1. **Dynamic HTML5/CSS Dashboards:** Transitioning from custom `Graphics2D` drawing to an embedded Chromium web view (JavaFX WebView) to support animated web dashboards.
2. **High-DPI Scaling:** Upgrading layout bounds to automatically adapt panels and layouts on high-resolution displays.
3. **Audio-visual Alerts:** Adding localized sound alerts and alert badges to notify users immediately when critical threats are blocked.

